



17 — Manual Reactivity: \$effect.root

ADVANCED LIFECYCLE

Q. How do you use reactivity like `$effect` outside of a component lifecycle (e.g., in a global store) without causing memory leaks?

ANSWER. Normally, effects live inside components so Svelte can automatically destroy them when the component unmounts. But if you create global state in a plain `.js` file and want an effect there (e.g., to auto-save to `localStorage`), Svelte blocks it to prevent memory leaks. You wrap it in `$effect.root`, which hands you a manual cleanup function to run when you're done.

```
import { $effect } from 'svelte';

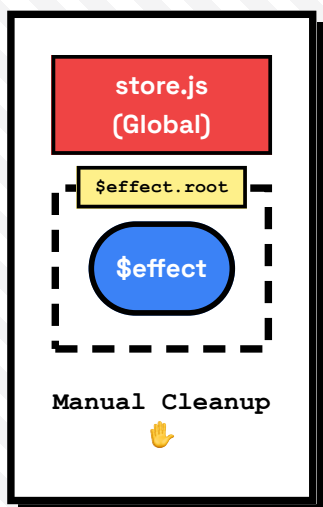
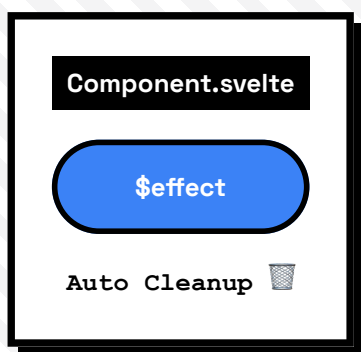
export function createGlobalLogger(stateValue) {
  // Returns a manual cleanup function
  const cleanup = $effect.root(() => {

    $effect(() => {
      console.log("State changed:", stateValue());
    });

  });

  return cleanup;
}
```

logger.js



① **Summary.** Normal effects die when their component unmounts. This is crucial because `$effect.root` lets you safely build global state managers or listeners in plain JavaScript files. It hands you the exact cleanup function you need to prevent memory leaks.